

MedRAG India

Production-Grade Medical Retrieval-Augmented Generation & Agentic Care Flow

1. Executive Summary & Design Principles

MedRAG India is an enterprise-grade medical assistant that combines **fine-tuned clinical adapters** with a multi-stage **hybrid RAG system**. The backend is orchestrating workflows via **LangGraph state machines**, ensuring structured, verifiable execution.

Core Architectural Principle: Patients receive educational context only, with strict refusals for direct diagnosis or prescribing queries. Licensed clinicians receive clinical decision support (e.g., differential considerations and drug interaction safety). The system uses pre-defined action routes and validation rules rather than allowing the LLM to write directly to DB stores or call external APIs dynamically. All access is audited.

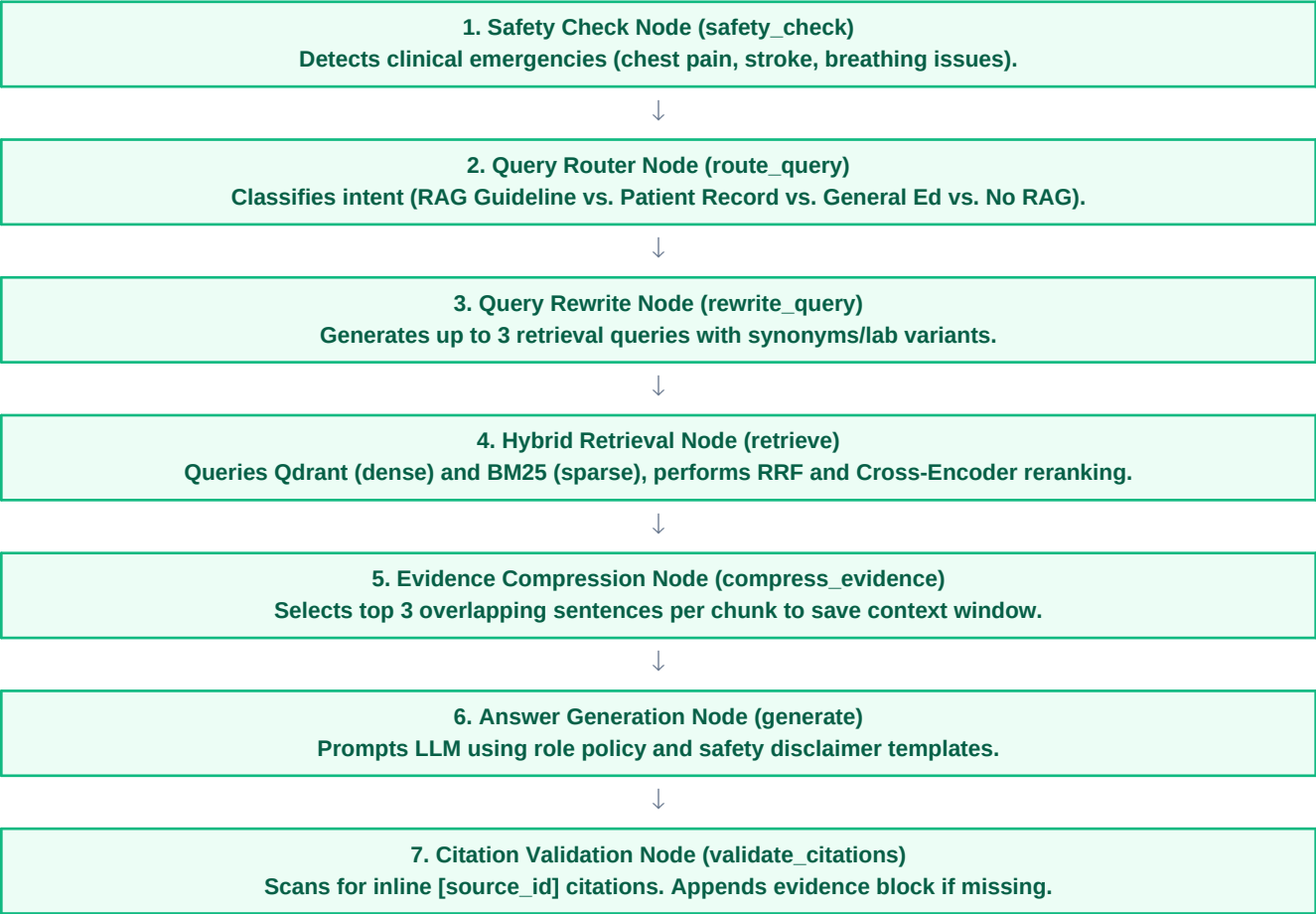
2. System Architecture & Entry Boundaries

When a request enters the FastAPI application, a series of validation, consent, and audit processes trigger before the RAG graph starts:

- **Authentication & Role Enforcement:** Resolves the user identity and validates permissions.
- **Compliance & Consent Checks:** The ComplianceService checks if a doctor has an active care-team membership and patient-granted consent. Patients can always self-access.
- **AI Policy Guardrails:** The AiPolicyService evaluates the query for diagnosis/prescription terms. If a patient violates the safety policy, an early refusal is issued. If not, it assigns the role-appropriate prompt instruction (educational vs decision support).
- **Audit Trail Logging:** The AuditService records the action, actor metadata, target patient, and query size into a hash-chained audit database.

3. Clinical Query State Machine Flow

After passing initial gate checks, the query triggers ClinicalRagGraph.invoke(). The graph coordinates execution across specialized nodes to ensure reliability and safety:



Node Safety Triggers & Exceptions

- **Emergency Escalation:** If safety_check flags an emergency, the graph immediately bypasses retrieval/generation nodes and jumps to finalize, returning a hardcoded recommendation to call emergency services (108/112).
- **Policy Refusal:** If the AiPolicyService flag was violated (e.g. patient requesting direct diagnosis), the graph routes directly to finalize to display the policy refusal warning, preventing LLM execution.
- **Skip Retrieval:** If the query router classifies the query as navigation or help (no_rag_needed), it skips retrieval nodes entirely and routes directly to the generator.

4. Care Coordination Agent Flow

The Care Coordination agent (CareCoordinationAgent) processes patient events, symptoms, and appointment requests. Crucially, it uses LangGraph to manage task flow but handles tool execution deterministically.

When symptom tracking or health calendars are updated, the graph proceeds through:

assess → schedule_yearly_scan / book_doctor_appointment / request_ambulance → log_action.

- **assess Node:** Classifies input symptoms using ClinicalSafetyService. If symptom severity is ≥ 9 or emergency terms match, it routes to emergency dispatch. If not, it routes to doctor appointment booking.
- **schedule_yearly_scan Node:** Automatically records a PatientCalendarEvent and a pending preventive HealthTask directly inside PostgreSQL using the SQLAlchemy session.
- **book_doctor_appointment Node:** Automatically records an Appointment table entry with urgency set relative to the symptom assessment score.
- **request_ambulance Node:** Calls the AmbulanceDispatchService (EMS boundary) to request emergency transport, saving an EmergencyDispatchRequest log with provider metadata.
- **log_action Node:** Records the final state, parameters used, and execution history into the database as a durable AgentActionLog, firing a secure audit event.

5. Tool-Calling Strategy: Bounded vs. Dynamic

A critical safety pattern in MedRAG India is that **the LLM does not execute direct tool-calling loops** (e.g., using OpenAI's tool-calling APIs or ReAct loops to call arbitrary functions).

Why Dynamic Tool Calling is Banned in Clinical Flows:

1. **Non-Deterministic Risk:** An LLM cannot be trusted to decide if an ambulance should be dispatched. Safety and writing database records must rely on strictly audited, deterministic rules.
2. **Data Leakage / Security:** Letting an LLM decide which SQL parameters or document directories to query creates high prompt-injection vulnerability.
3. **Separation of Concerns:** The LLM is confined to text-to-text processing nodes (classifying query intent, rewriting query variants, and generating evidence-grounded answers). Data retrieval (Qdrant/SQL) and database writing are executed by robust, structured Python code in separate graph nodes.

6. Key Subsystems and Mechanics

6.1 Hybrid Retrieval Pipeline

The HybridMedicalRetriever is designed to maximize recall and precision for Indian medical contexts:

1. **Dense Semantic Search:** Embeds queries using BGE-M3. Queries the Qdrant database applying patient-isolation visibility filters (e.g. only public guides or documents specifically matching the current patient's ID).
2. **Sparse Keyword Search:** Runs BM25Okapi keyword searches over the retrieved candidates. This prevents the semantic model from missing exact chemical/drug names (e.g. *Sildenafil*, *Metformin*) or laboratory metrics (e.g. *HbA1c*).
3. **Reciprocal Rank Fusion (RRF):** Merges dense and sparse lists using Reciprocal Rank Fusion, ensuring chunks that score highly in both algorithms appear at the top.
4. **Cross-Encoder Reranker:** Scores query-chunk pairs via a reranker model to filter out irrelevant context.
5. **Parent Context Expansion & Compression:** Retrieves parent context windows, then uses EvidenceCompressionService to pull out only the top 3 sentences containing maximum overlap with the user's query, preserving model context limits.

6.2 Data Security & Regulatory Safeguards

The system complies with HIPAA and DPDP regulations through strict data restrictions:

- **Download Restrictions:** A hard API policy restricts clinical staff from executing raw document downloads. Only the patient owner is permitted to download binary reports.
- **PHI Scrubbing:** The PrivacyService automatically redacts common identifiers (Aadhaar, ABHA cards, emails, phone numbers) before writing conversation data to the database.
- **Answer Tracing:** The AnswerTraceService records the execution profile (model name, prompt version, retrieved context snippets, latency, and user role) for full clinical audibility.
- **Cache Headers:** Responses enforce strict HTTP headers to prevent local browser caching of health data:

```
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Expires: 0
```